```
import numpy as np
import random
import matplotlib.pyplot as plt

# Grid: 0 = free space, 1 = obstacle
grid = np.array([
    [0,0,0,0,0],
    [0,1,0,0,0],
    [0,0,0,1,0],
    [0,0,0,0,0],
    [0,0,0,0,0]
])

start = (0, 0)
goal = (4, 4)


ants = 20
iterations = 100

alpha = 1
beta = 2
evaporation = 0.5

rows, cols = grid.shape

# Initialize pheromone
pheromone = np.ones((rows, cols))

def get_neighbors(pos):
    r, c = pos

    moves = [
        (r-1, c),
        (r+1, c),
        (r, c-1),
        (r, c+1)
    ]

    return [
```

```
        (x, y) for x, y in moves
        if 0 <= x < rows and 0 <= y < cols
        and grid[x][y] == 0
    ]

def heuristic(pos):
    return 1 / (
        abs(pos[0] - goal[0]) +
        abs(pos[1] - goal[1]) + 1
    )

best_path = None
best_length = float('inf')

# ACO algorithm
for iteration in range(iterations):

    paths = []

    for ant in range(ants):

        current = start
        path = [current]
        visited = {current}

        while current != goal:

            neighbors = [
                n for n in get_neighbors(current)
                if n not in visited
            ]

            if not neighbors:
                break

            probabilities = []

            for n in neighbors:
                p = pheromone[n] ** alpha
                h = heuristic(n) ** beta
```

```
                probabilities.append(p * h)

            probabilities = np.array(probabilities)
            probabilities /= probabilities.sum()

            current = random.choices(
                neighbors,
                weights=probabilities,
                k=1
            )[0]

            path.append(current)
            visited.add(current)

        if current == goal:
            paths.append(path)

            length = len(path) - 1

            if length < best_length:
                best_length = length
                best_path = path

    # Pheromone evaporation
    pheromone *= (1 - evaporation)

    # Pheromone update
    for path in paths:
        length = len(path) - 1

        for r, c in path:
            pheromone[r][c] += 1 / length

print("Best Path:", best_path)
print("Minimum Path Length:", best_length)
```

For Display Grid

```
# Display grid and path

plt.figure(figsize=(6, 6))

plt.imshow(grid, cmap="Greys")

if best_path:
    x = [p[1] for p in best_path]
    y = [p[0] for p in best_path]

    plt.plot(x, y, 'ro-', linewidth=2, markersize=8)

plt.scatter(start[1], start[0], color='green', s=150, label='Start')
plt.scatter(goal[1], goal[0], color='blue', s=150, label='Goal')

plt.title("ACO Robot Path Planning")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

